

Design and Development of Games

Internship Report

*Submitted in partial fulfilment of the requirements for the award
of the Degree of*

Master of Computer Application

by

Saurabh Yadav

21234757048

Under the Guidance of

Associate Professor

Mr. Mantosh Biswas



**DEPARTMENT OF COMPUTER SCIENCE
UNIVERSITY OF DELHI DELHI-110007, INDIA
July,2024**

ABSTRACT

This report documents the comprehensive internship experience at Keywords Studios, where I engaged in developing various game prototypes, including classic and strategy games. The primary objective was to gain practical experience in game development by working on different aspects such as physics simulation, collision detection, advanced game logic, and multiplayer architecture.

Throughout the internship, I contributed to the creation of an SFML Archery Game, a multiplayer Snake Game, and a multiplayer Chess Game.

The SFML Archery Game focused on simulating realistic physics for arrow trajectories, enhancing my skills in game mechanics implementation. Simple and Fast Multimedia Library (SFML) was chosen for its simplicity and efficiency in handling graphics rendering and window management, which are crucial for creating visually appealing and responsive games.

The Snake Game involved creating a multiplayer version with real-time communication and synchronization between players. ReactJS was used for building the game's user interface due to its component-based architecture and efficient rendering capabilities. Socket.io was utilized to enable real-time, bidirectional communication between the server and clients, ensuring smooth interactions and game state updates. Firestore, a NoSQL cloud database from Firebase, was used for player authentication and storing game data due to its scalability and ease of integration.

The Chess Game required implementing complex game logic, move validation, and multiplayer functionality using a C++ backend server and a Node.js server. C++ was chosen for its performance and control over system resources, which is essential for handling complex game logic and ensuring efficient server operations. Node.js was used for managing multiplayer interactions and real-time updates, leveraging its non-blocking, event-driven architecture to handle multiple simultaneous connections effectively.

The diverse projects provided a broad learning experience, allowing me to explore various genres and mechanics in game development. This internship equipped me with essential skills and knowledge to navigate future game development endeavours, emphasizing the importance of collaboration, communication, and personal growth in a professional setting.

UNDERTAKING

I declare that the work presented in this report titled “*Design and Development of Games*”, submitted to the Department of Computer Science, University of Delhi, New Delhi, for the award of the Master of Computer Application degree, is our original work. I have not plagiarized or submitted the same in part or full to any university or institution for the award of any degree or diploma. In case this undertaking is found incorrect, I accept that our degree may be unconditionally withdrawn.

Date: 29/07/2024

A handwritten signature in blue ink, appearing to read 'S. Yadav', with a stylized flourish at the end.

Saurabh Yadav

21234757048

Place: Delhi



सगणक विज्ञान विभाग
DEPARTMENT OF COMPUTER SCIENCE
दिल्ली विश्वविद्यालय, दिल्ली – 110007 (भारत)
UNIVERSITY OF DELHI, DELHI – 110 007 (INDIA)

CERTIFICATE

I hereby certify that the work which is being presented in this MCA Internship Project report entitled “*Design and Development of Games*”, in partial fulfillment of the requirements for the award of the Master of Computer Application is an authentic record of my own work carried out during a period from 9th February, 2024 to 5th July, 2024 under the supervision of mentor Dr. Mantosh Biswas, Associate Professor, Department of Computer Science, University of Delhi, Delhi and Industrial mentor Mr. Ajay Shekhar, Technical Director, Keywords Studios, Gurgaon. The matter presented in this project report has not been submitted for the award of any other degree elsewhere.

Date: 29/07/2024

Place: Delhi

Saurabh Yadav
21234757048

This is to certify that the above statement made by the candidates is correct to the best of my knowledge.

Mr. Ajay Shekhar
(Industrial Supervisor)
Technical Director
Keywords Studios
Gurgaon – 122101

Dr. Mantosh Biswas
(Departmental Supervisor)
Associate Professor
Department of Computer Science
University of Delhi, Delhi - 110007

ACKNOWLEDGEMENT

I am profoundly grateful to the **Department of Computer Science, University of Delhi**, for providing me with the opportunity to pursue an internship at **Keywords Studios**.

I extend my sincere gratitude to **Prof. Mantosh Biswas** and the esteemed faculty members of the Department of Computer Science for their unwavering support and guidance. Their efforts in facilitating this internship have been crucial in bridging my academic knowledge with practical industry experience.

I would also like to express my heartfelt thanks to my supervisor at Keywords Studios **Mr. Ajay Shekhar** for his guidance and support during my internship. I benefited greatly by working under their guidance. It was this effort for which I am able to develop a detailed insight on this subject and special interest to study further. Their encouragement motivation and support have been invaluable throughout my internship.

My appreciation also extends to my colleagues and team members at Keywords Studios Their collaboration, encouragement, and willingness to share their expertise greatly contributed to a productive and enjoyable internship.

Finally, I am deeply grateful to my family and friends for their unwavering support and encouragement throughout this journey. Their belief in me has been a continuous source of motivation and strength.

My apologies and heartfelt gratitude to all who have assisted me yet have not been acknowledged by name.

Saurabh Yadav
21234757048

Table of Content

Introduction

- 1.1. Company Overview
- 1.2. Project Background
- 1.3. Problem Definition
- 1.4. Objectives

2. Proposed Solution

- 2.1. General Game Architecture
 - 2.1.1. Game Loop
 - 2.1.2. Game State
 - 2.1.3. Game Logic
 - 2.1.4. Graphic Engine
 - 2.1.5. Physics Engine
 - 2.1.6. Networking

3. Implementation Approach

- 3.1. Language Used
- 3.2. Libraries and Tools
- 3.3. Database Integration

4. Implementation Plan in Detail

- 4.1. Archery Game
 - 4.1.1. Design and Mechanics
 - 4.1.2. Physics Simulation
 - 4.1.3. Technical Implementation
- 4.2. Snake Game
 - 4.2.1. Design and Mechanics
 - 4.2.2. Technical Implementation
- 4.3. Chess Game
 - 4.3.1. Design and Mechanics
 - 4.3.2. Technical Implementation

5. Execution and Result

- 5.1. Archery Game
- 5.2. Snake Game
- 5.3. Chess Game

6. Discussion

- 6.1. Technical Skill Development
- 6.2. Collaboration and Communication
- 6.3. Personal Growth

7. Conclusion

- 7.1. Summary of Experience
- 7.2. Future Aspect

8. Reference

List of Figures

Figure 1: Three main phases of a Game Loop

Figure 2: Overview of General Physics Engine

Figure 3: Matchmaking Room Example

Figure 4: Types of Bounding Boxes

Figure 5: Connection and Components in Client-Server Architecture

Figure 6: Client-Server Chess Architecture

Figure 7: Archery Game Simulation

Figure 8: Snake Game Login / Signup page

Figure 9: Snake game mode option

Figure 10: Multiplayer Game Instance

Figure 11: Chess signin,signup page

Figure 12: Chess Game option Page

Figure 13: Chess Game View

Figure 14: Chess Game Menu

Figure 15: Chess Game Leaderboard

Figure 16: Chess Game Personal Stats

Figure 17: Chess game AI Assistant

CHAPTER 1

INTRODUCTION

1.1 Company Overview

Keywords Studios Plc (Keywords) is a video game industry services company that provides technical and creative services to the global video games industry. Keywords Studios offers a complete outsourced game art, engineering, testing, audio, and localization service for all console, PC, handheld, and mobile content. It also offers a range of services such as art, game development, audio, functionality quality assurance, localization, localization quality assurance, marketing services and player support. Founded in 1998 by Giorgia Guastalla and Teresa Luppino, the company has over 70 studios in 25+ countries and a team of over 13,000. Keywords also provides additional services that include technical support, customer service, performance testing, embedded services and game science service. It offers its services to the video games industry. The company operate in Canada, Ireland, India, Japan, Singapore, Italy, the US and the UK. Keywords is headquartered in Dublin, Ireland.

Just a few of Keyword's credits:

- Halo Infinite
- Zelda: Breath of the Wild
- Star Wars Battlefront
- League of Legends
- Assassin's Creed: Syndicate
- The outer Worlds
- Clash of clans
- Injustice 2
- Call of Duty: Black Ops

1.2 Project Background

During my internship, I engaged in the development of various game prototypes to deepen my understanding of game development. The projects included both classic and strategy games, each designed to address different aspects of game creation.

Developed titles such as the SFML Archery Game and Snake Game, focusing on fundamental game development concepts like physics simulation, collision detection, and real-time interactions. Worked on more complex projects like the Chess Game, which required advanced game logic, move validation, and multiplayer architecture.

This diverse approach allowed me to explore different genres and mechanics, providing a broad learning experience. The projects involved working on multiple prototypes, which helped me gain practical skills and insight into various game development challenges.

1.3 Problem Definition

The problem definition was to gain a full understanding of game development by working on prototypes across classic and strategy genres. This approach aimed to build a solid foundation in core game development concepts while introducing the complexities of multiplayer functionality.

Classic games presented challenges in mastering fundamental mechanics like graphics rendering, collision detection, and real-time input handling. Adding multiplayer features introduced complexities related to real-time communication and synchronization between players.

Strategy games demanded delving into advanced game logic, including rule enforcement and move validation. Developing a multiplayer version added the challenge of understanding and implementing server-client architecture for real-time interactions.

By tackling these diverse challenges, the project aimed to equip me with the skills and knowledge to navigate a wide range of game development endeavors in the future.

1.4 Objective

The primary objective of the internship program was to provide hands-on experience in game development by actively participating in the creation of both classic and strategy games. This involved contributing to every stage of the development lifecycle, from design and development to testing and iteration, ensuring a comprehensive understanding of game development principles and practices. The specific goals were:

1. **SFML Archery Game:** Develop a single-player game that simulates realistic physics for arrow trajectories and collision detection, enhancing skills in physics simulation and game mechanics implementation.
2. **Snake Game:** Create a multiplayer version of the classic Snake game, focusing on real-time communication and synchronization between players to ensure smooth interactions and game state management.
3. **Chess Game:** Implement a fully functional multiplayer Chess game, utilizing a C++ backend server for complex game logic and move validation, and a Node.js server for managing multiplayer interactions, thereby providing a seamless and interactive user experience.

Chapter 2

Proposed Solution

2.1 General Game Architecture

2.1.1 Game Loop

Among the architectural components the single most important one is game loop functioning as the central mechanism that drives the game's logic and renders its graphics. It's essentially a continuous cycle of events that governs the progression of the game from one frame to the next. In simple term it is a loop similar to a 'for' or a 'while' loop that we are aware of. In each loop the updating of suppose a character's position or in more depth the change of the pixel intensity to simulate a continuous motion.

It typically consists of three main phases:

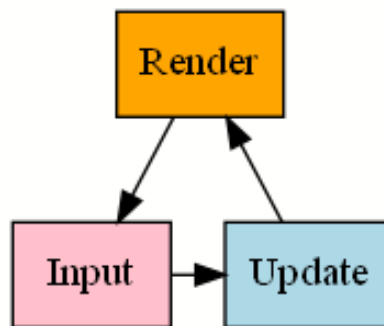


Fig. 01: Three main phases of a Game Loop

- a) **Input:** During the run time of a game, the user will give input be it via keyboard or mouse or other, the game captures user input and processes it to determine the player's actions, such as moving characters, triggering events, or interacting with objects in the game world. It will look for an event in each iteration of the game loop.
- b) **Update:** After processing input, the game proceeds to update its internal Game state (mentioned later) based on the user input and any other dynamic factors. This includes updating the positions and properties of game objects, handling collisions, executing game logic (e.g., AI behaviour), and managing any ongoing processes or timers.
- c) **Render:** Once the game state is updated, in the rendering phase, the updated game world is displayed to the player. This involves drawing graphics, text, and other

visual elements to the screen, often using techniques like sprites, textures, shaders, and 3D models.

2.1.2 Game State

The collective information of everything happening within the game world at a specific moment in time. It includes aspects like positions, properties, and behaviours of all game objects, the status of the player characters, environmental conditions, and other data that help to accurately describe the current state of gameplay. It serves as an important part for managing the flow of gameplay.

2.1.3 Game Logic

Game logic dictates the rules, mechanics, and interactions that govern gameplay. Everything from player actions, win conditions, and environmental effects. Implementing efficient game logic makes engaging, balanced, and immersive gaming experiences. It's the invisible hand guiding players through challenges and rewarding their decisions within the virtual world.

2.1.4 Graphics Engine

A graphics engine is responsible for rendering visual elements in a video game. It handles tasks such as geometry processing, lighting, shading, and texture mapping to generate realistic or stylized images. Graphics engines utilize hardware acceleration and optimization techniques to achieve smooth and immersive visuals. They support various rendering techniques, including rasterization, ray tracing, and particle effects, to bring game worlds to life. The efficiency and capabilities of a graphics engine directly impact the quality and performance of the gaming experience.

2.1.5 Physics Engine

It is responsible for simulating the physical behaviour and interaction between objects within the game world. It gives the object realistic effects like friction, gravity etc. It's a software system that simulates the laws of physics and calculates the movements and interactions/collision of objects in a game environment. We simulated our own physics engine in the Archery game discussed further.

The following diagram shows a high-level overview of the general procedure of a physics engine.

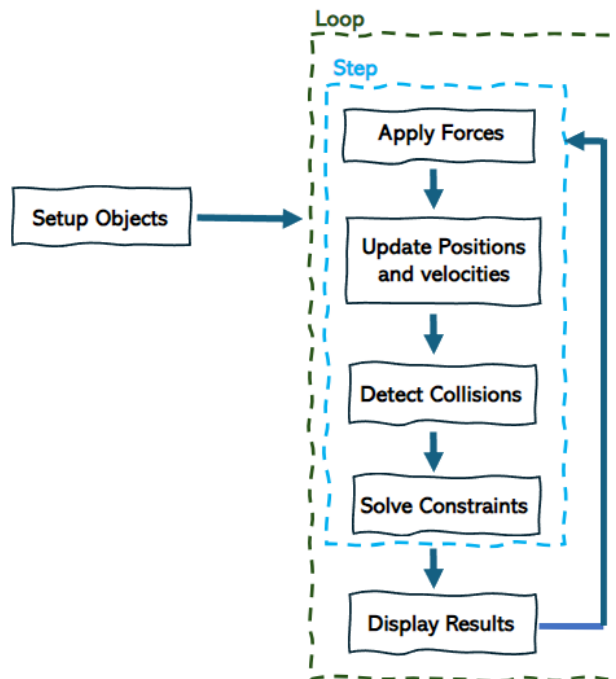


Fig 02: Overview of general physics Engine

2.1.6 Networking

Networking is also very important in games particularly in enabling multiplayer experiences, where players from different locations can interact, compete, or cooperate within the same virtual game world. It involves managing communication between multiple game instances or players over a network (implemented in snake and chess game discussed later in the report), facilitating features such as matchmaking, synchronization, and various multiplayer game modes.

These criteria are briefly discussed below to highlight the importance of networking,

- i) **Matchmaking:** Matchmaking algorithms are used to pair players together based on various criteria such as skill level, geographical location, preferred game modes, and player preferences. To ensure enjoyable gameplay experience alike players are intended to be matched with each other.

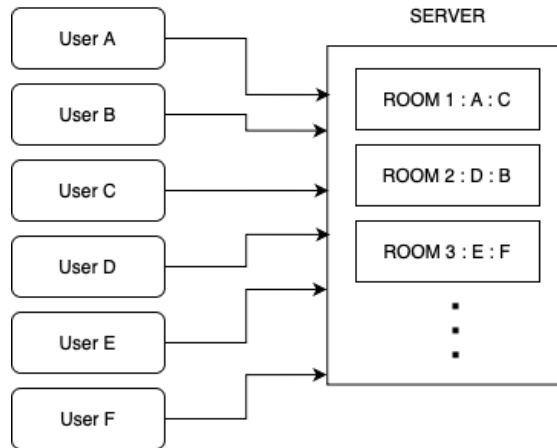


Fig 03: Matchmaking Room Example

- ii) **Communication Protocols:** Various communication protocols facilitate the data exchange between game clients and servers. Common protocols include TCP (Transmission Control Protocol) and UDP (User Datagram Protocol), while UDP is often preferred for real-time multiplayer games due to its lower overhead and better support for fast-paced, latency-sensitive gameplay, TCP is used for more reliable data transmission in situations where packet loss is less tolerable.
- iii) **Synchronization:** To maintain consistency between different game instances and ensuring that all players experience the same game state we synchronize the local game state in client with the global game state from the server. This involves synchronizing the positions, actions, and interactions of players and game objects across the network. Techniques such as client-server architecture are commonly used to achieve synchronization in multiplayer games.

Chapter 3

Implementation Approach

3.1 Languages Used

- i) **C++:** language was used as most of the current major game development studios use C++ and also most of the game engines are made to be used with C++, it is imperative for game developer to have a good grasp on it.
It was used in Archery game to write the complete game logic and in the Chess game to create the C++ backend server handling the complex chess logic.
- ii) **JavaScript:** programming language is commonly used for creating dynamic and interactive web applications. It runs directly in the browser, making it ideal for handling client-side tasks.
I utilized it to implement game mechanics, user controls, and real-time updates for both the Snake and Chess games.
- iii) **Node.js:** is a cross-platform JavaScript runtime environment that allows code written in JavaScript to run outside of a web browser. It helped to develop the server-side logic for the multiplayer functionality in both my chess and snake games. This enabled real-time communication and data synchronization between players, creating a seamless multiplayer experience.

3.2 Libraries and Tools

- i) **ReactJS :** is a JavaScript library for building user interfaces (UIs) on the web. React is a declarative, component based library that allows us to build reusable UI components and It follows the Virtual DOM approach.
It is used in the Snake Game to design the frontend.
- ii) **SFML :** Simple and Fast Multimedia Library is designed to provide a simple interface for game development and has a lot of features like graphics rendering window management, input handling, networking.
It was used in the archery game to draw and render bow and target.
- iii) **Socket.io :** is a real-time library that enables two-way communication between web servers and browsers. In our chess and snake multiplayer games, it allows players to send information (like moves in chess or snake direction changes) to the server and receive updates (like opponent's moves or snake positions) instantly, creating a seamless multiplayer experience.

Additionally, Socket.io provides rooms, which are virtual spaces where you can group players. This is particularly helpful for multiplayer games, as you can create separate rooms for different chess matches or snake games.

3.3 Database Integration

- i) **Firestore:** Firebase provides Firestore which is flexible, scalable NoSQL cloud database, built on Google Cloud infrastructure, to store and sync data for client- and server-side development.

It is used in snake and chess games to persist and store player details and game information.

Chapter 4

Implementation Plan in Detail

4.1 Archery Game

4.1.1 Design and Mechanics

- i) **Objective:** The objective of the Archery game is to shoot arrows and hit the target. Players aim by adjusting the bow's angle and releasing the mouse click to launch the arrow.
- ii) **Gameplay Overview:** The game consists of a single-player mode where the player aims at a stationary target.
- iii) **Core Gameplay Loop:**
 - a) Player positions the bow by moving the mouse.
 - b) On mouse click, the arrow is launched with a speed based on the mouse drag distance.
 - c) Arrow trajectory is calculated based on launch angle and gravity.
 - d) Collision detection checks if the arrow hits the target.
 - e) The game provides feedback (points awarded) based on whether the target is hit.
- iv) **Scoring:** Points are awarded based on the distance between the arrow and the centre of the target.

4.1.2 Physics simulation

1. Calculating Projectile Angle and Rotation:

Formula -

$$\theta = \tan^{-1} \left(\frac{y_2 - y_1}{x_2 - x_1} \right)$$

In our case, (x1,y1) represents the arrow's position (arrow.getPosition()) and (x2, y2) represents the mouse position (mousePosition). The resulting value (θ) represents the direction the arrow should be launched in.

Rotation Adjustment: Based on projectile angle, the arrow also needs to be rotated.

2. Setting Projectile Speed:

$$speed = k . (initMouseX - CurrMouseX)$$

The difference between the initial X position of the mouse and the current mouse position is used as a basis, likely scaled by a constant factor K for balancing purposes (in our case 0.5f). This approach ensures that a larger drag (i.e., a greater difference in mouse position) results in a higher projectile speed.

3. Projectile Trajectory Simulation:

Formula - The core physics behind the trajectory simulation relies on the following formulas for horizontal and vertical displacement over time (t):

$$\Delta x = speed * t * \cos(\theta)$$

$$\Delta y = speed * t * \sin(\theta) - \frac{1}{2} * g * t^2$$

Where:

- speed is the launch speed.
- θ is the launch angle.
- g is the acceleration due to gravity (approximately $9.8 \frac{m}{s^2}$).

4.1.3 Technical Implementaion

C++ was used to write the game logic and the SFML library it has these classes called sprites `sf::Sprite` that are used to include 2D images or texture `sf::Texture` that can be displayed on the screen, spirits are often used to represent characters, objects, or other visual elements within a game. I have used these sprite to render the images of bow and target.

All object interaction in games are in some way or other handled using collision, without this objects will pass through each other, we want the bodies to stop after getting closer by a very small distance.

The concept of bounding box is used for detecting collision, or more accurately we can say it puts an upper threshold on the collision. Though there are concepts like broad phase and narrow phase, where broad phase find object pairs that might be colliding, then narrow phase detects among them who are colliding. But these concepts are not needed in this game, a simple bounding box collision detection does the work.

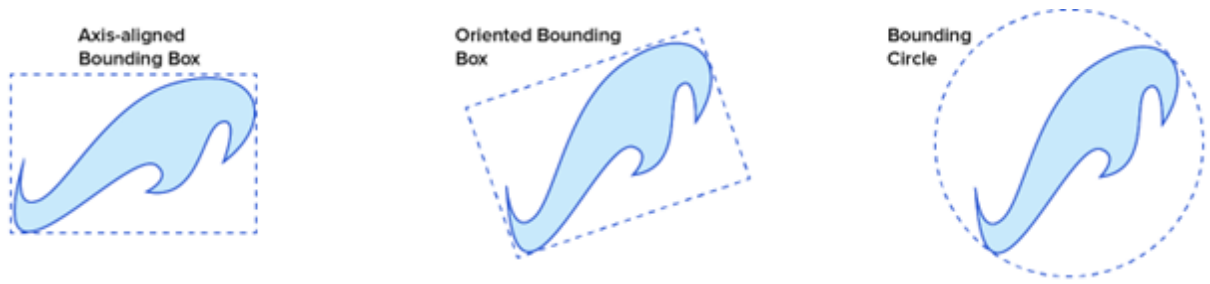


Fig 04: Types of Bounding boxes

Bounding Box - A bounding box is a simplified geometric shape used to enclose complex objects in games. It aids in collision detection, making computations more efficient. Common types include axis-aligned (AABB) and oriented (OBB) bounding boxes. They are easy to compute and manipulate, enhancing performance in game development.

This game used bounding box to detect collision among the arrow and target.

4.2 Snake Game

4.2.1 Design and Mechanics

- i) **Objective:** The objective of the Multiplayer Snake game is to grow your snake longer than your opponents by strategically collecting food items while avoiding collisions. Be the last snake remaining to win the game!
- ii) **Gameplay Overview:** The game is a real-time multiplayer experience where players control a snake on a shared platform.
- iii) **Core Gameplay Loop:** Each player controls their snake's movement using keyboard input (e.g., arrow keys)
 1. The snake moves continuously in the chosen direction (up, down, left, right).
 2. The server keeps track of the location and direction of all snakes.
 3. The server broadcasts the updated positions of all snakes to all connected clients.
 4. Clients update their local game state based on the received information.
 5. Food items are randomly spawned on the platform by the server.
 6. When a snake's head collides with a food item:
 - a. The snake's length increases by one unit.

- b. The server broadcasts the updated snake length to all clients.
 - c. The food item is respawned at a new random location by the server.
- 7. The game ends for a snake if it collides with:
 - a. The boundary of the platform.
 - b. Its own body.
 - c. Another player's snake body.
- 8. The server keeps track of surviving players and determines the winner based on the longest remaining snake.

4.2.2 Technical Implementation

- a) **Framework Choice:** We chose ReactJS for the frontend game logic for rendering.
- b) **Player Authentication and Database:** Firebase was used for authentication and Firestore for storing player data like high scores and usernames. Players register with an email, username, and password. The username is used for the leaderboard.
- c) **Multiplayer Architecture:** We created a client-server architecture using WebSockets with Node.js and Socket.io for low latency communication. The game logic was moved to the server to ensure real-time updates and synchronization. When a player changes direction, the input is sent to the server, which updates the game state and sends it back to the client, ensuring immediate synchronization.

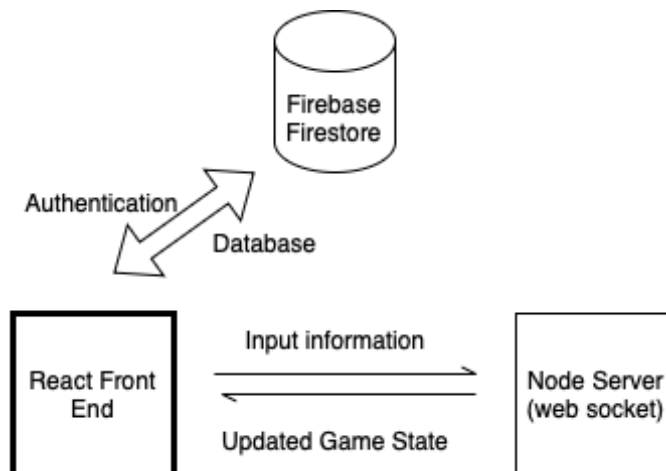


Fig 05: Connection and components in client - server architecture

4.3 Chess Game

4.3.1 Design and Mechanics

- a) **Gameplay Overview:** CHESS has a board on which the game is played, it has 8 row and 8 columns making 64 checked cells, on which our piece can move, although there are many different openings and school of attacks on chess, the piece have basic movements. Players are of either white or black colour.

Each player has 16 pieces, among them –
8 Pawn, 2 Knight, 2 Bishop, 2 Rook, A king and A Queen

- b) **Objective:** The goal is to checkmate the opponent's king by placing it in a position where it's under direct attack (check) and cannot escape.

4.3.2 Technology Used

- **Three-Layer Architecture:**

- a) **Client:** Simple HTML, CSS, JavaScript that would use Socket.io to form connection with the server. Same concept of room is also used here to make it so that player can multiplayer.
- b) **Node.js Server:** It uses socket.io for the connection, node has the responsibility of authenticating player, creating room, formatting the JSON with appropriate command-id (discussed later) for the identification of service required from the C++ logic server. To connect to the C++ logic server Node server makes http get request. As this is a turn based game, the latency is not that much of an issue, and HTTP requests are appropriate for it. Node uses socket.io for client to server, Axios for C++ communication.
- c) **C++ Server:** This server is for game logic, it keeps the game state for all the rooms, all the functionalities like valid moves of a piece are provided by this server to the node, all check logic and check mate logic.
C++ uses boost's ASIO library for asynchronous server, and nlohmann's json library for processing JSON received from node.

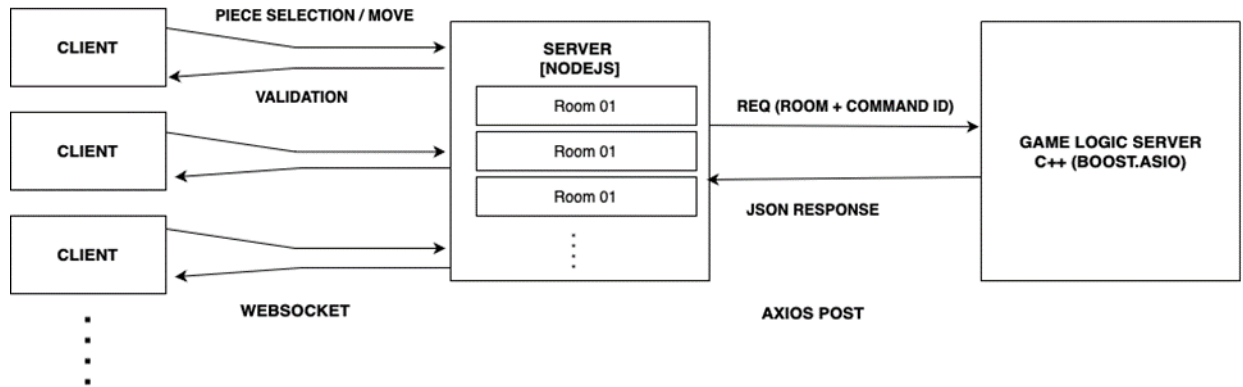


Fig 06 : Client Server Chess Architecture

- Communication Protocol:** The communication between client and node server is via socket.io's event – listener model, but such a communication is not possible via http post requests that we are doing with the C++ logic server, so to tackle that with every post request that we do to the C++ server, in the JSON part we add a field as “command id” that is then resolved in the C++, to decide what the node server wants to C++ to do with this post request, e.g. for command id are create_room, valid_move, update_position, validate_check etc.
- Player Authentication and Database:** We used Firebase, to add authentication and used firebase's Firestore for storing player data like user name, scores, match detail, etc.
- AI Integration** To elevate the player experience, We integrated a conversational AI assistant into our Chess game, named Professor Magnus Veritas, a fictional chess grandmaster and professor, this AI provides players with strategic guidance and insights as they play. Players can interact with the AI by posing questions related to chess strategy, analysing past moves, or seeking advice on improving their tactics.

Chapter 5

Execution and Results

5.1 Archery Game



Fig 07: Archery Game Simulation

5.2 Snake Game

When player first goes to the website, they get a login or signup screen where they have to register themselves with *email id*, a *username*, and *password*. This username will later be used in the leader board.

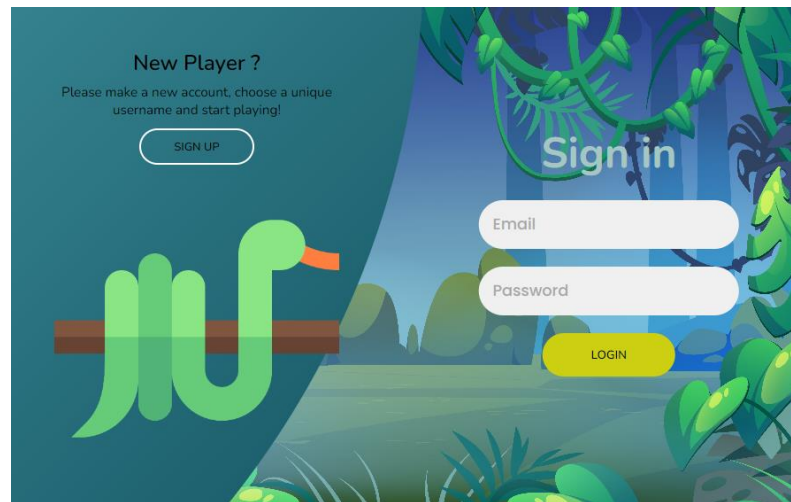


Fig 08 : Snake Game Login / Signup page

After authentication is done user is redirection to options page, to choose from single or multiplayer, if multiplayer then would they like to create a room or join a room.

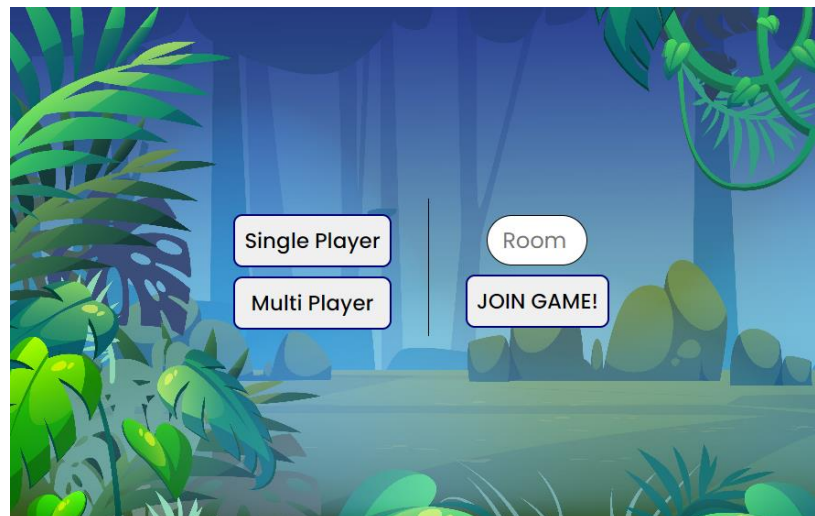


Fig 09 : Snake game mode option

When the game was created by the first player by clicking “Multi Player” button, the server generates a room-id a five-letter random combination of alpha numeric, this code has to inputted by the second player to join the same room. Second player joins the room, the game gets activated and can then be played again and again,

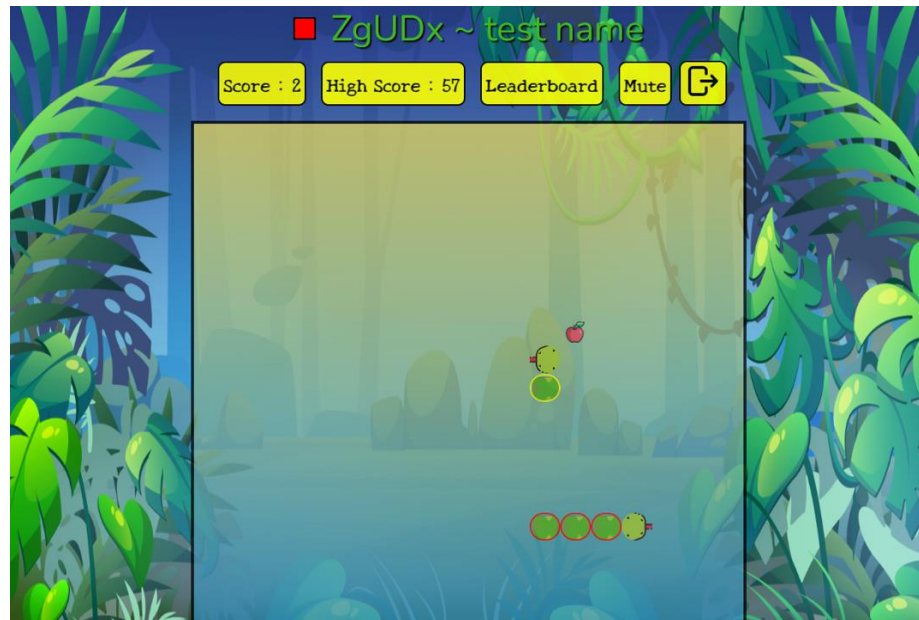


Fig 10: Multiplayer Game Instance

5.3 Chess Game

When the user open the address for the client, we get routed to the Signin/Signup Page for user authentication.



Fig 11: Chess signin,signup page

After successful authentication user get routed to the front game option page, which gives us a choice of whether we want to Create a new game with the option of choosing our chess piece (Black / White) or Join a existing game with the roomId.



Fig 12: Chess Game option Page

Choosing any option will lead you to the game floor,

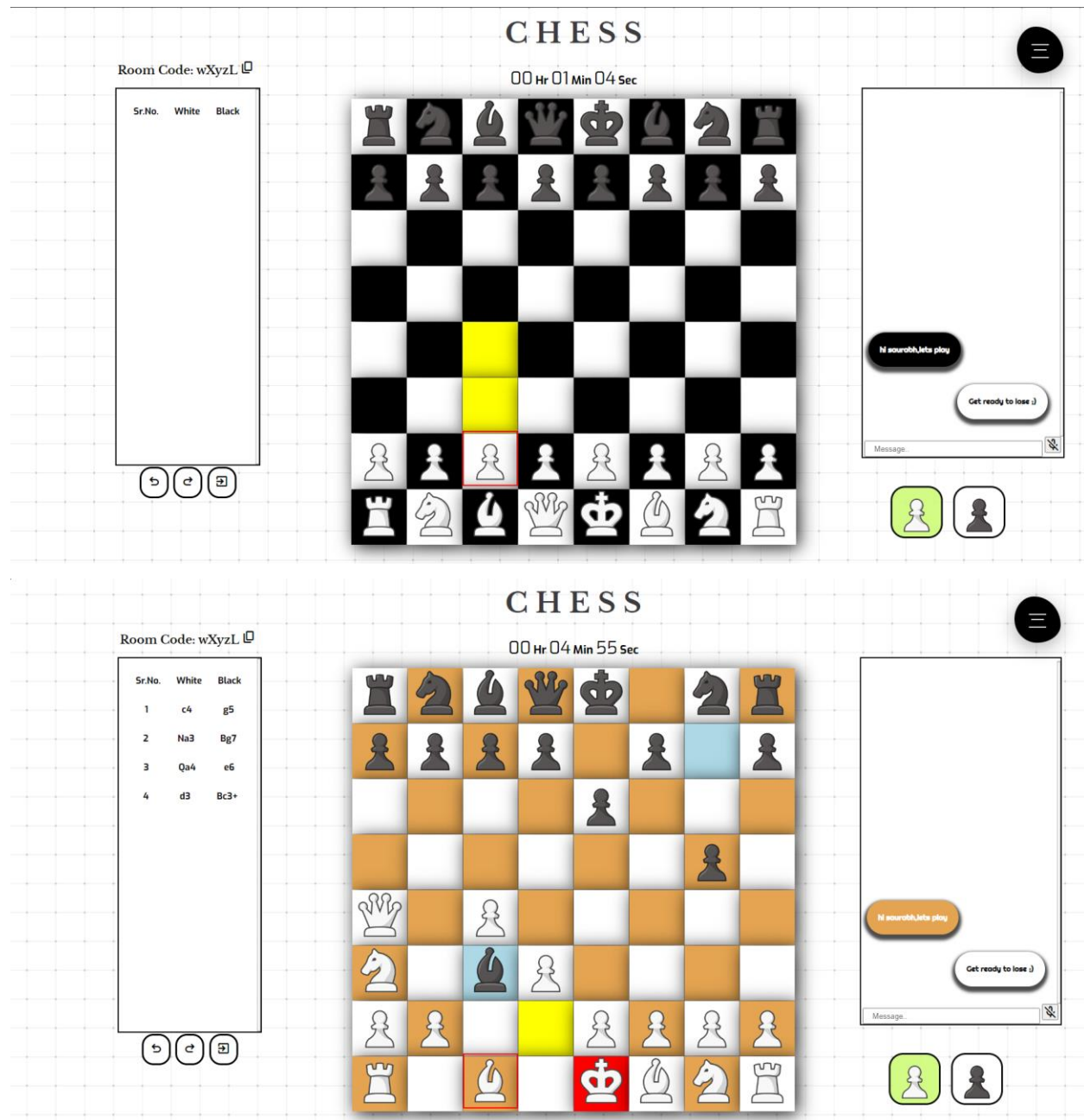


Fig 13 :Chess Game View

On the left side of the board we can see the move board, which displays what move was played by each player throughout the game, and on the right side is a chat window where the players in the game can talk to each other and interact.

The Game also provides a navigation drawer,where player can :

- Change the game theme
- Restart the game

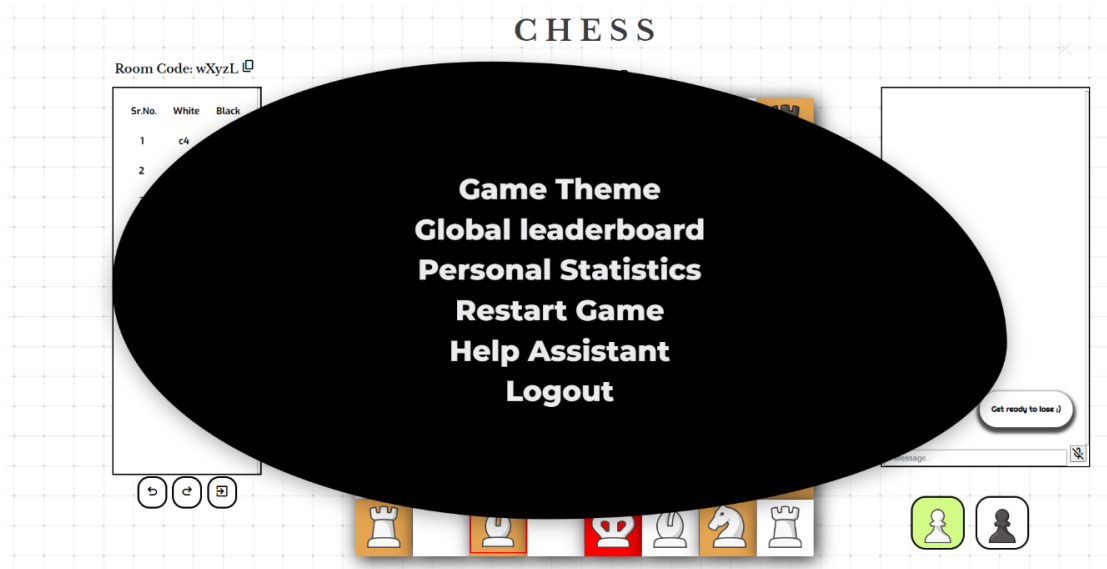


Fig 14 : Chess Game Menu

- Check the global leaderboard

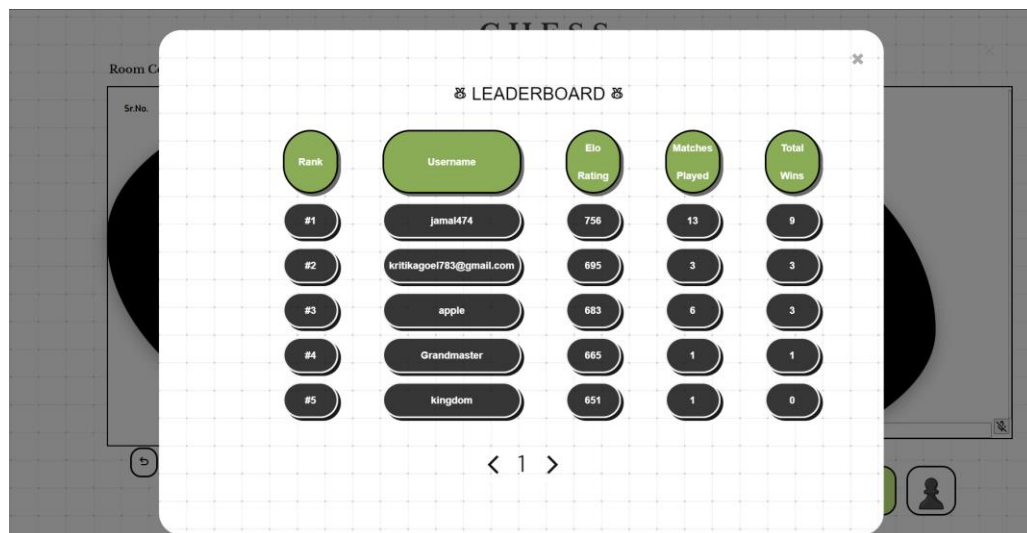


Fig 15 : Chess Game Leaderboard

- Check his/her personal statistics.

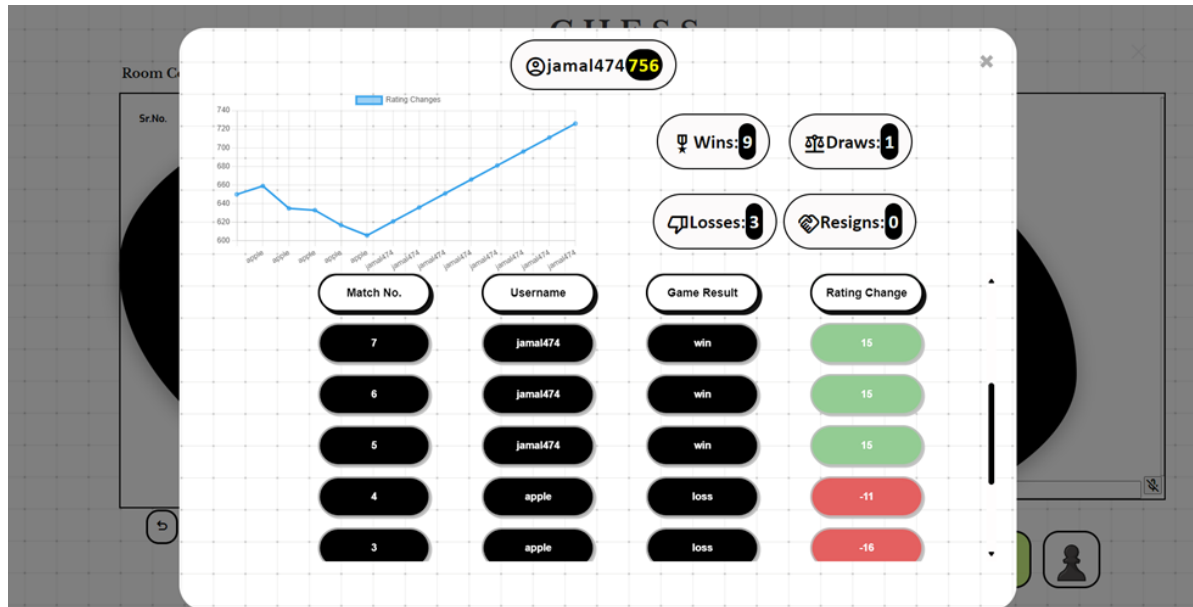


Fig 16 : Chess Game Personal Stats

- AI Assistant: Conversational AI Integration

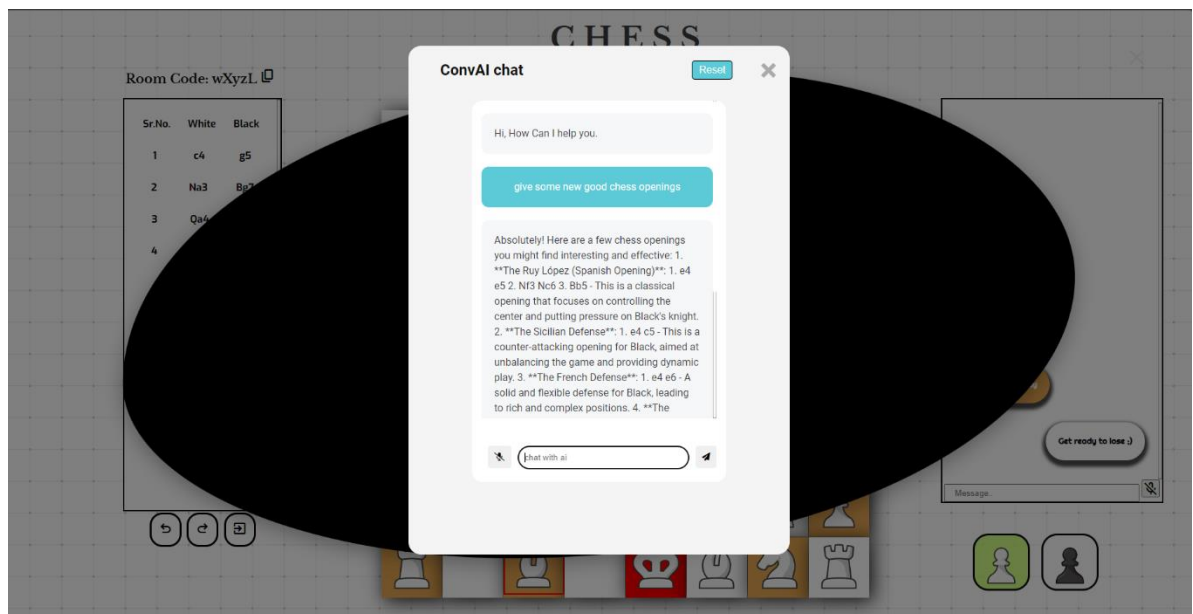


Fig 17 : Chess game AI Assistant

Chapter 6

Discussion

6.1 Technical Skill Development

The internship significantly enhanced my technical skills, including:

- i) **Backend Development:** Improved my skills in using Node.js and Socket.io and C++ for building scalable server-side applications and handling real-time communication and writing complex game logic.
- ii) **Database Management:** Developed expertise in using Firebase Firestore for real-time database management, including user authentication and storing player data.
- iii) **Real-Time Communication:** Gained experience in implementing WebSockets for low-latency, bidirectional communication between clients and servers.
- iv) **Multiplayer Game Logic:** Learned to implement room-based logic for multiplayer interactions and synchronization of game state across clients.

6.2 Collaboration and Communication

- i) **Team Collaboration:** Coordinated with team members to ensure smooth integration of frontend and backend components and adherence to project timelines.
- ii) **Feedback and Reviews:** Participated in code reviews and incorporated feedback to enhance code quality and project outcomes.

6.3 Personal Growth

The internship contributed to my personal growth in several ways:

- i) **Confidence:** Gained confidence in applying technical skills to real-world problems and presenting solutions effectively.
- ii) **Time Management:** Improved my ability to manage time and prioritize tasks, balancing multiple responsibilities and deadlines.
- iii) **Professionalism:** Developed a professional attitude towards work, including attention to detail, responsibility, and commitment to quality.
- iv) **Adaptability:** Enhanced my ability to adapt to new technologies and frameworks quickly, demonstrating a willingness to learn and grow.

Chapter 7

Conclusion

7.1 Summary of Experience

Throughout this internship I came to know about many topics from Computer Science that are used in the gaming industry, I got practical knowledge as to how to approach a project, how planning is done, how writing Design Documents, and Technical Design Documents help in keeping the correct expectations from the project and how POC (Proof of Concept) are made before the actual game or project is started.

Collaborating and working closely with other team members, I learned how to leverage diverse skills and perspectives to solve complex problems more efficiently.

I also dived into game development for the web. This involved understanding the nuances of network programming and real-time data synchronization, which are essential for creating responsive multiplayer games.

7.2 Future Aspect

This internship provided a foundation for further development of the created games. Here are some potential enhancements:

i) **Archery Game:**

- a) Improved physics simulation for more realistic arrow trajectories.
- b) Difficulty levels and environmental variations for a richer experience.
- c) Advanced graphical effects and sound design for enhanced quality.

ii) **Snake Game:**

- a) Optimized real-time synchronization for reduced latency.
- b) New game modes, power-ups, and obstacles for increased variety.
- c) Social features (chat, friend lists) to create player community.

iii) **Chess Game:**

- a) Implementation of the castling feature for a complete experience.
- b) Enhanced AI offering more challenging opponents.
- c) Tutorials and practice modes to assist new players.

Chapter 8

Reference

[1] Souto, N. (2015, March 2). *Video Game Physics Tutorial - Part II: Collision Detection for solid objects*. Toptal Engineering Blog. <https://www.toptal.com/game/video-game-physics-part-ii-collision-detection-for-solid-objects>

[2] Souto, N. (2015a, January 22). *Video Game Physics Tutorial - Part I: An Introduction to Rigid Body Dynamics*. Toptal Engineering Blog. <https://www.toptal.com/game/video-game-physics-part-i-an-introduction-to-rigid-body-dynamics>

[3] Tyler, D. (2023, July 28). How to use physics in your game design. *Video Game Design and Development*. <https://www.gamedesigning.org/learn/game-physics/>

[4] Burzynski, Denny & Cooper, Kelly & Focht, Veronika. (2023). *Mathematics for Game Developers Student's Edition*.